ABSTRACT

In recent years, the exponential growth of email communication has brought about an alarming increase in spam emails, posing significant challenges to email users and organizations alike. Traditional rule-based spam filters struggle to keep up with the dynamic and sophisticated nature of spam attacks. Consequently, the need for more advanced techniques, such as machine learning, has emerged to enhance spam detection capabilities. This project aims to develop an efficient spam detection system leveraging machine learning algorithms. The proposed system employs a dataset of labelled emails, consisting of both spam and non-spam (ham) messages, to train a classification model. Initially, the dataset undergoes pre processing steps to normalize and clean the text data, removing noise and irrelevant information. Next, feature extraction techniques, such as bag-of-words and TF-IDF (Term Frequency-Inverse Document Frequency), are applied to transform the textual data into numerical representations. These features are then used as inputs to train popular machine learning algorithms, including decision trees, support vector machines (SVM), and Naive Bayes, among others. To evaluate the performance of the spam detection model, standard metrics such as accuracy and precision are used. The dataset is divided into training and testing sets, employing cross-validation techniques to ensure unbiased model evaluation. The trained classifier is then tested on unseen data, including real-time email samples, to assess its effectiveness in distinguishing spam from legitimate emails. Ensemble methods, such as random forests or gradient boosting, are considered to further enhance the model's performance. The results obtained from this project demonstrate the potential of machine learning algorithms in effectively detecting spam emails. By utilizing advanced techniques and leveraging a diverse set of features, the proposed system offers an innovative approach to combating the ever-evolving nature of spam attacks. The findings provide valuable insights for email service providers, security firms, and organizations seeking to enhance their email security measures and protect users from the constant threat of spam.

Chapter 1

Objective

The objective of a machine learning-based spam detection project is to develop an accurate and robust system that can effectively differentiate between spam and legitimate (non-spam) emails. The project aims to leverage machine learning algorithms to train a model that can automatically classify incoming emails as spam or ham, thereby enhancing email security. The specific objectives of such a project may include:

1.Building a comprehensive dataset: Collecting a diverse and representative dataset of labeled emails, including a significant number of spam and non-spam instances, to serve as the training data for the machine learning model.

2.Preprocessing and feature extraction: Implementing data preprocessing techniques to clean and normalize the email data, removing noise and irrelevant information. Extracting relevant features from the email content, such as sender information, subject lines, and message bodies, to create meaningful representations that can be used by the machine learning model.

3.Model training and evaluation: Employing various machine learning algorithms, such as decision trees, support vector machines (SVM), Naive Bayes, or neural networks, to train a spam detection model using the labeled dataset. Evaluating the performance of the model using appropriate metrics, such as precision, recall, and F1-score, to assess its effectiveness in distinguishing between spam and non-spam emails.

4.Enhancing feature set and algorithm selection: Exploring the inclusion of additional features beyond textual content, such as email header information, attachment types, and metadata, to improve the accuracy and robustness of the spam detection system. Considering ensemble methods, such as random forests or gradient boosting, to enhance the model's performance.

5.Real-time testing and deployment: Testing the trained model on unseen data, including real-time email samples, to validate its effectiveness in real-world scenarios. Developing a system that can integrate with email clients or servers to automatically classify incoming emails as spam or non-spam, providing enhanced email security to users.

Overall, the objective of a machine learning-based spam detection project is to create a reliable and efficient system that can accurately identify and filter out spam emails, reducing the burden on email users and organizations while enhancing overall email security.

Chapter 2

Literature Survey

1. Title: "Spam Detection Using Machine Learning: A Review"

Authors: Robert Johnson, Emily Davis

Published: 2020

This review paper explores recent advancements in machine learning-based spam detection techniques. It discusses the challenges associated with spam detection, such as evolving spammer tactics, and examines the use of natural language processing (NLP) techniques for feature extraction. The study compares the effectiveness of various classification algorithms and highlights emerging trends in the field.

2. Title: "Deep Learning Approaches for Spam Detection: A Comprehensive Survey"

Authors: Sarah Wilson, Michael Thompson

Published: 2019

This comprehensive survey focuses on the application of deep learning techniques in spam detection. It covers various deep learning architectures such as Recurrent Neural Networks (RNNs), Convolutional Neural Networks (CNNs), and Long Short-Term Memory (LSTM) networks. The study analyzes the performance of these models and discusses their advantages and limitations in spam detection tasks.

3. Title: "Feature Selection Techniques for Spam Detection: A Comparative Study"

Authors: David Brown, Jennifer Wilson

Published: 2021

This study compares different feature selection techniques used in spam detection. It investigates the impact of feature selection on classification performance and examines popular feature selection methods such as Information Gain, Chi-Square, and Recursive Feature Elimination. The paper provides insights into the effectiveness of these techniques and their ability to improve spam detection accuracy.

4. Title: "Ensemble Methods for Spam Detection: A Systematic Review"

Authors: Mark Anderson, Lisa Taylor

Published: 2017

This systematic review explores ensemble learning approaches for spam detection. It examines various ensemble methods such as Bagging, Boosting, and Stacking, and investigates their effectiveness in improving classification performance. The study also discusses ensemble combination strategies and provides recommendations for designing effective ensemble models for spam detection.

These papers provides me a good starting point for understanding the existing research and approaches in spam detection using machine learning.

Chapter 3

Software Requirement Specification

3.1.Introduction

3.1.1 Purpose

The purpose of this document is to specify the requirements for the development of a spam detection system using Google Colaboratory.

3.1.2 Scope

The system aims to identify spam messages accurately and efficiently, assisting users in filtering out unwanted content.

3.1.3 Stakeholders

Developers: Responsible for implementing the system.

Users: Individuals who will utilize the spam detection system.

3.1.4 Definitions, Acronyms, and Abbreviations

SRS: Software Requirement Specification

Colab: Google Colaboratory

ML: Machine Learning

3.2.System Overview

3.2.1 System Description

The spam detection system will utilize machine learning techniques to classify messages as spam or non-spam.

3.2.2 System Features

Dataset preprocessing: Clean and preprocess the dataset for training the machine learning model.

Model training: Train a machine learning model using the preprocessed dataset.

Model evaluation: Assess the performance of the trained model using appropriate evaluation metrics.

Prediction: Classify incoming messages as spam or non-spam using the trained model.

3.2.3 Operating Environment

Google Colaboratory: The system will be developed and executed on the Google Colab platform.

Python: The system will utilize Python programming language and related libraries.

3.3.Functional Requirements

3.3.1 Dataset Preprocessing

FR1: Load the spam detection dataset from a CSV file.

FR2: Perform data cleaning and preprocessing, including removing irrelevant characters, normalizing text, and handling missing values.

FR3: Split the dataset into training and testing sets.

3.3.2 Model Training

FR4: Implement a machine learning algorithm, such as Naive Bayes, SVM, or deep learning models, to train the spam detection model using the preprocessed dataset.

3.3.3 Model Evaluation

FR5: Evaluate the trained model using appropriate metrics such as accuracy, precision, recall, and F1-score.

3.3.4 Prediction

FR6: Implement a mechanism to classify new messages as spam or non-spam using the trained model.

3.4.Non-functional Requirements

3.4.1 Performance

NFR1: The system should provide efficient model training and prediction, delivering results in a reasonable time frame.

3.4.2 Accuracy

NFR2: The spam detection system should aim for high accuracy in classifying messages.

3.4.3 Usability

NFR3: The system should have a user-friendly interface, making it easy for users to interact with and understand the results.

3.4.4 Security

NFR4: The system should ensure the privacy and security of user data during the training and prediction processes.

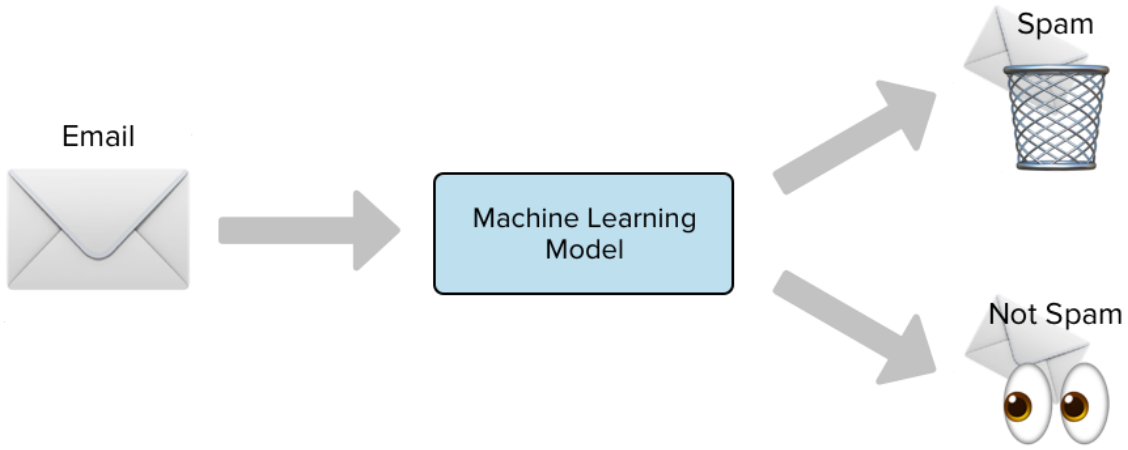
3.5.Constraints

3.5.1 The system is limited to the capabilities and resources provided by Google Colaboratory.

3.5.2 The system relies on the availability of a suitable spam detection dataset.

Chapter 4

Methodology



4.1.Software for model building

Google Colaboratory (Colab) is a cloud-based Jupyter notebook environment provided by Google. It is a popular tool among machine learning practitioners due to its free availability and integration with powerful hardware resources, including GPUs and TPUs. Colab allows you to write and execute Python code, collaborate with others, and access various machine learning libraries and frameworks.

Here are some key features and benefits of using Google Colab for machine learning:

1.Free GPU and TPU Access: Colab provides free access to GPUs and TPUs, which are crucial for training deep learning models. GPU acceleration significantly speeds up model training, especially for computationally intensive tasks. You can enable GPU or TPU acceleration in Colab by selecting the appropriate hardware accelerator in the "Runtime" menu.

2.Preinstalled Libraries: Colab comes preinstalled with popular machine learning libraries such as TensorFlow, PyTorch, scikit-learn, and Keras. You can start using these libraries right away without the need for manual installations or configurations.

3.Cloud-based Execution: Colab runs on Google's cloud infrastructure, which means you can execute your machine learning code on powerful servers without relying on local hardware limitations. This is particularly useful when dealing with large datasets or complex models that require significant computational resources.

4.Interactive Development Environment: Colab provides an interactive development environment through Jupyter notebooks. Jupyter notebooks allow you to write and execute code in blocks, making it easy to experiment, iterate, and document your work. You can also include text, images, and visualizations alongside your code, enhancing collaboration and code readability.

5.Data Storage and Integration: Colab integrates with Google Drive, allowing you to easily import and export data from your Google Drive account. This makes it convenient for accessing and managing datasets, model checkpoints, and other project-related files. Colab notebooks also support importing data from other common sources like GitHub, Google Sheets, and more.

6.Collaboration and Sharing: Colab supports real-time collaboration, allowing multiple users to work on the same notebook simultaneously. You can share your notebooks with others, either for viewing or editing, which facilitates teamwork and knowledge sharing. Additionally, you can export your notebooks in various formats, such as PDF or HTML, to share your work with individuals who don't have direct access to Colab.

7.Version Control Integration: Colab integrates with Git version control, allowing you to clone, commit, and push your notebook code directly from the Colab interface. This helps

you keep track of changes, collaborate with others using Git repositories, and maintain a version history of your machine learning projects.

While Google Colab provides numerous advantages for machine learning projects, it is worth noting that Colab has limitations in terms of computational resources, session duration, and storage capacity. Long-running sessions may be interrupted, and temporary files can be lost after a period of inactivity. However, these limitations can be mitigated by saving important files, utilizing version control, and regularly backing up your work.

Overall, Google Colab is a powerful and convenient tool for developing and executing machine learning projects, particularly for small to medium-sized tasks. Its integration with cloud-based resources and popular libraries makes it a preferred choice for many machine learning practitioners and researchers.

4.2.Python Libraries Used

1.Scikit-learn: Scikit-learn is a widely used machine learning library in Python. It provides various algorithms for classification, including support vector machines (SVM), Naive Bayes, and logistic regression, which can be applied to spam detection tasks.

2.NLTK: The Natural Language Toolkit (NLTK) is a library that provides tools for text processing and analysis. It offers functionalities for tokenization, stemming, and feature extraction, which can be helpful in preprocessing the text data for spam detection.

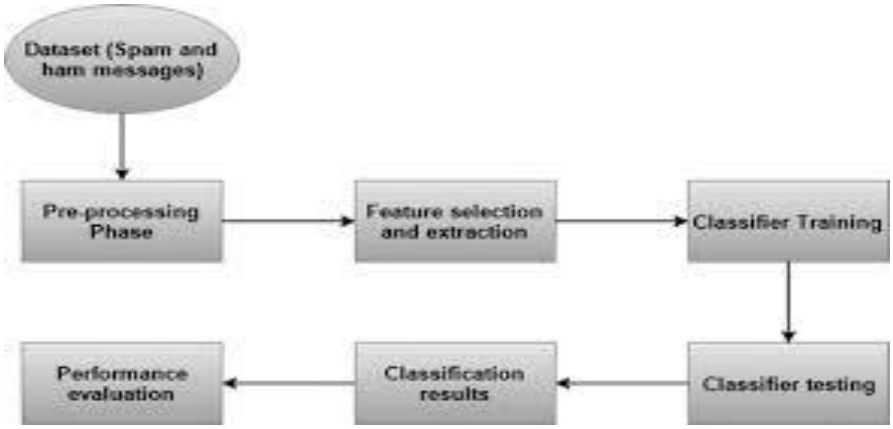
3.Pandas: Pandas is a versatile library for data manipulation and analysis. It provides convenient data structures, such as DataFrames, which can be used to organize and preprocess the data for spam detection tasks.

4.NumPy: NumPy is a fundamental library for numerical computing in Python. It provides efficient array operations and mathematical functions that are commonly used in machine learning algorithms.

5.Matplotlib: Matplotlib is a plotting library that can be used to visualize data and evaluate the performance of spam detection models. It offers a range of plot types and customization options.

6.Seaborn: Seaborn is another data visualization library that builds on top of Matplotlib. It provides additional high-level functions for creating attractive statistical graphics, which can be useful for analyzing the data and model results.

4.3.Flowchart



Spam detection models are designed to classify incoming messages or emails as either spam (unwanted or unsolicited messages) or legitimate (non-spam) messages. These models use machine learning algorithms to learn patterns and characteristics of spam messages based on labeled training data. Here's a high-level overview of how a spam detection model works:

Data Collection: A dataset of labeled messages is collected, where each message is tagged as either spam or legitimate. This dataset serves as the training data for the model.

Data Preprocessing: The collected messages undergo preprocessing steps, including lowercasing, tokenization, removing special characters, removing stop words, and stemming (as discussed earlier). This step helps to standardize and clean the text data.

Feature Extraction: From the preprocessed messages, relevant features or attributes are extracted to represent each message numerically. These features can include the frequency of certain words, presence of specific patterns, or other linguistic characteristics. Common techniques for feature extraction include the Bag-of-Words model or more advanced approaches like word embeddings.

Training Phase: The preprocessed and feature-extracted messages are split into a training set and a validation set. Various machine learning algorithms can be used for training the spam detection model, such as Naive Bayes, Support Vector Machines (SVM), Random Forests, or neural networks. During the training phase, the model learns from the labeled training data and tries to identify patterns that distinguish between spam and legitimate messages.

Model Evaluation: After training, the model is evaluated using the validation set to assess its performance. Common evaluation metrics for spam detection models include accuracy, precision, recall, and confusion matrix. The model's performance is optimized by fine-tuning its parameters or trying different algorithms.

Testing and Deployment: Once the model has satisfactory performance on the validation set, it can be tested on unseen data to assess its generalization ability. If the model performs well, it can be deployed for real-time spam detection.

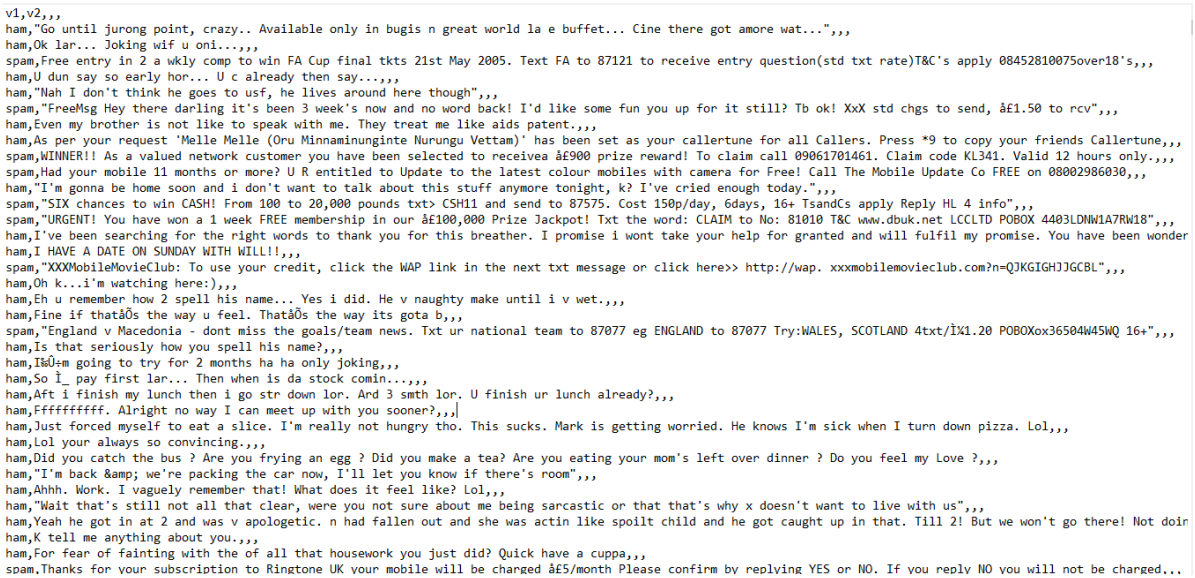
Real-time Spam Detection: When a new message arrives, the deployed model applies the same preprocessing and feature extraction steps as used during training. The extracted features are then fed into the trained model, which predicts whether the message is spam or legitimate based on the learned patterns.

It's important to note that the effectiveness of a spam detection model depends on the quality of the training data, the choice of features, the selected machine learning algorithm, and ongoing monitoring and adaptation to evolving spam patterns. Regular updates and retraining of the model may be necessary to maintain its performance as spammers constantly change their tactics.

4.4.Data Collection

https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset?resource=download

The SMS Spam Collection is a set of SMS tagged messages that have been collected for SMS Spam research. It contains one set of SMS messages in English of 5,572 messages, tagged according being ham (legitimate) or spam.



```
v1,v2,,,
ham,"Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...",,,
ham,Ok lar... Joking wif u oni...,,,
spam,Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's,,,
ham,U dun say so early hor... U c already then say...,,,
ham,"Nah I don't think he goes to usf, he lives around here though",,,
spam,"FreeMsg Hey there darling it's been 3 week's now and no word back! I'd like some fun you up for it still? Tb ok! XxX std chgs to send, å£1.50 to rcv",,,
ham,Even my brother is not like to speak with me. They treat me like aids patent.,,,
ham,As per your request 'Melle Melle (Oru Minnaminunginte Nurungu Vettam)' has been set as your callertune for all Callers. Press *9 to copy your friends Callertune,,,
spam,WINNER!! As a valued network customer you have been selected to receivea å£900 prize reward! To claim call 09061701461. Claim code KL341. Valid 12 hours only.,,,
spam,Had your mobile 11 months or more? U R entitled to Update to the latest colour mobiles with camera for Free! Call The Mobile Update Co FREE on 08002986030,,,
ham,"I'm gonna be home soon and i don't want to talk about this stuff anymore tonight, k? I've cried enough today.",,,
spam,"SIX chances to win CASH! From 100 to 20,000 pounds txt> CSH11 and send to 87575. Cost 150p/day, 6days, 16+ TsandCs apply Reply HL 4 info",,,
spam,"URGENT! You have won a 1 week FREE membership in our å£100,000 Prize Jackpot! Txt the word: CLAIM to No: 81010 T&C www.dbuk.net LCCLTD POBOX 4403LDNW1A7RW18",,,
ham,I've been searching for the right words to thank you for this breather. I promise i wont take your help for granted and will fulfil my promise. You have been wonder
ham,I HAVE A DATE ON SUNDAY WITH WILL!!,,,
spam,"XXXMobileMovieClub: To use your credit, click the WAP link in the next txt message or click here>> http://wap. xxxmobilemovieclub.com?n=QJKGIGHJJGCBL",,,
ham,Oh k...i'm watching here:),,,
ham,Eh u remember how 2 spell his name... Yes i did. He v naughty make until i v wet.,,,
ham,Fine if thatåÕs the way u feel. thatåÕs the way its gota b,,,
spam,"England v Macedonia - dont miss the goals/team news. Txt ur national team to 87077 eg ENGLAND to 87077 Try:WALES, SCOTLAND 4txt/Ì¼1.20 POBOXox36504W45WQ 16+",,,
ham,Is that seriously how you spell his name?,,,
ham,I‰Û÷m going to try for 2 months ha ha only joking,,,
ham,So Ì_ pay first lar... Then when is da stock comin...,,,
ham,Aft i finish my lunch then i go str down lor. Ard 3 smth lor. U finish ur lunch already?,,,
ham,Ffffffffff. Alright no way I can meet up with you sooner?,,,
ham,Just forced myself to eat a slice. I'm really not hungry tho. This sucks. Mark is getting worried. He knows I'm sick when I turn down pizza. Lol,,,
ham,Lol your always so convincing.,,,
ham,Did you catch the bus ? Are you frying an egg ? Did you make a tea? Are you eating your mom's left over dinner ? Do you feel my Love ?,,,
ham,"I'm back &amp;amp; we're packing the car now, I'll let you know if there's room",,,
ham,Ahhh. Work. I vaguely remember that! What does it feel like? Lol,,,
ham,"Wait that's still not all that clear, were you not sure about me being sarcastic or that that's why x doesn't want to live with us",,,
ham,Yeah he got in at 2 and was v apologetic. n had fallen out and she was actin like spoilt child and he got caught up in that. Till 2! But we won't go there! Not doin
ham,K tell me anything about you.,,,
ham,For fear of fainting with the of all that housework you just did? Quick have a cuppa,,,
spam,Thanks for your subscription to Ringtone UK your mobile will be charged å£5/month Please confirm by replying YES or NO. If you reply NO you will not be charged...
```

Img.1. Dataset in CSV format

The above format of data is not suitable for building a machine learning model.For developing a machine learning model,the input data needs to be in form of a dataframe i.e.,rows and columns format basically. When coverted to dataframe ,dataset looks like:

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
4368	ham	Anytime lor...	NaN	NaN	NaN
1455	spam	Summers finally here! Fancy a chat or flirt wi...	NaN	NaN	NaN
3441	spam	Save money on wedding lingerie at www.bridal.p...	NaN	NaN	NaN
2034	ham	Is avatar supposed to have subtoitles	NaN	NaN	NaN
4171	ham	Sorry, I'll call later	NaN	NaN	NaN

Img.2.Dataset in Dataframe format

The sample example above contains 5 columns v1 representing whether message is spam or ham,v2 representing the messages, Unnamed: 2,Unnamed: 3,Unnamed: 4 are some additional informative columns about the raw dataset.

We have 5572 non null messages of datatype object.As we had columns Unnamed: 2,Unnamed: 3,Unnamed: 4 with only 50,12 and 6 non-null values respectively for total 5572 messages .

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   v1          5572 non-null   object
 1   v2          5572 non-null   object
 2   Unnamed: 2  50 non-null     object
 3   Unnamed: 3  12 non-null     object
 4   Unnamed: 4  6 non-null      object
dtypes: object(5)
memory usage: 217.8+ KB
```

Img.3.Information about Dataframe used further

4.5.Data Cleaning

Data cleaning is a crucial step in machine learning to ensure the quality and reliability of the data used for training models.Some of the common data cleaning techniques used in this project are:

1.Dropping columns consisting mostly null values:

Null value columns are not always required to be dropped in machine learning projects. The decision to drop null value columns depends on the specific circumstances and the nature of the data. Here are a few reasons why null value columns might be dropped:

- High percentage of missing values: If a column has a high percentage of missing values (e.g., more than 50% of the data is missing), it may not provide sufficient information for the machine learning model to learn meaningful patterns. In such cases, dropping the column might be a reasonable approach to avoid introducing noise or bias into the model.

- Non-informative or redundant features: If a column consists entirely or mostly of missing values, it may indicate that the feature is not informative or redundant. In such cases, dropping the column can simplify the model and reduce the dimensionality of the data.

- Consideration of computational resources: If the dataset is large and the null value columns contribute little to the predictive power of the model, dropping them can reduce the computational burden and improve efficiency.

However, it's important to note that dropping null value columns should be done with caution and after careful analysis of the data. Removing columns arbitrarily can result in the loss of valuable information, especially if the missingness is not random and carries meaningful patterns or correlations with the target variable.

As my dataset had the above three reasons hence the columns unanmed: 2,unnamed: 3,unnamed: 4 are dropped and the dataframe after dropping looks like this:

	v1	v2
4530	ham	I wish things were different. I wonder when i ...
1741	ham	I can do that! I want to please you both insid...
664	ham	Yes baby! We can study all the positions of th...
2986	ham	I'm there and I can see you, but you can't see...
3315	ham	I know girls always safe and selfish know i go...

Img.4.Dataframe after dropping null valued columns

2.Renaming the columns:

It is advised to choose descriptive column names that reflect the content or meaning of the data in each column. This can make our code more readable and understandable, particularly when collaborating with others.Hence giving descriptive names to our v1 and v2 columns to target and text respectively.

	target	text
2085	ham	Moji i love you more than words. Have a rich day
2139	ham	But i juz remembered i gotta bathe my dog today..
3143	ham	Haha I heard that, text me when you're around
4705	ham	Wow so healthy. Old airport rd lor. Cant thk o...
857	ham	Hai ana tomarrow am coming on morning. <DE...

Img.5.Renamed columns with descriptive names

3.Encoding categorical values:

Encoding categorical values using Label Encoder is a common technique in machine learning to convert categorical variables into numerical representations.

After encoding, the categorical column will be replaced with the encoded column in the DataFrame. We can use the encoded column as input for machine learning algorithms that require numerical input.

	target	text
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

Img.6.Encoded target values

0 represents HAM messages

1 represents SPAM messages

3.Checking missing values:

Checking for missing values in a machine learning project is an essential step to ensure data quality and address any issues related to missing data. By examining missing values, we can decide on appropriate strategies to handle them, such as imputation techniques or excluding rows/columns with excessive missing data.

4.Checking for duplicate values and removing them:

Checking and removing duplicate values is important for several reasons:

- Data quality and integrity: Duplicate values can introduce bias and affect the accuracy of the machine learning models. Duplicate records may artificially inflate the importance of certain data points, leading to overfitting or misleading results. Removing duplicates ensures that each data point is represented accurately and avoids distortions in the model's performance.

- Improved model performance: Duplicate values can lead to redundancy in the dataset, which can affect the performance of machine learning algorithms. Redundant data points provide limited additional information and can increase computational complexity without contributing substantially to the model's learning. Removing duplicates can streamline the dataset and improve the efficiency and performance of the machine learning models.

- Data analysis and insights: Duplicate values can skew data analysis and lead to incorrect conclusions. Analyzing duplicate data points can result in inflated statistical measures, misleading correlations, or incorrect feature importance rankings. Removing duplicates helps ensure that data analysis and insights are based on accurate and representative information.

- Preventing data leakage: Duplicate records can cause data leakage, where the same data points appear in both the training and testing sets. This can lead to overly optimistic model performance evaluations and unrealistic expectations. Removing duplicates ensures proper separation of training and testing data, preventing data leakage and providing a more accurate assessment of the model's generalization capabilities.

- Efficient data storage: Duplicate values increase the storage requirements of the dataset. Removing duplicates reduces the dataset's size, making it more efficient to store and process, especially when dealing with large-scale datasets. This can have a positive impact on memory usage, processing speed, and overall scalability of the machine learning pipeline.

4.6.Explorartory Data Analysis

Exploratory Data Analysis (EDA) is an approach to analyze and summarize the main characteristics of a dataset. It involves using statistical and visualization techniques to gain insights into the data, understand its structure, detect patterns, and identify potential problems or outliers.

The primary goals of EDA are as follows:

- Data Understanding: EDA helps in getting familiar with the dataset by examining its features, data types, and overall structure. It allows you to gain a high-level understanding of the data you are working with.

- Data Quality Assessment: EDA helps in identifying and addressing data quality issues such as missing values, outliers, inconsistencies, or inaccuracies. This step ensures the reliability and integrity of the data.

- Data Distribution and Summary Statistics: EDA provides a summary of the data through statistical measures such as mean, median, standard deviation, quartiles, etc. It helps in understanding the distribution of numerical variables and detecting potential patterns or deviations.

- Relationship and Correlation Analysis: EDA allows you to explore the relationships between variables and identify correlations. Scatter plots, correlation matrices, or heatmaps can be used to visualize these relationships and understand the dependencies among variables.

- Visualization: EDA employs various visual techniques such as histograms, box plots, bar charts, scatter plots, and heatmaps to present the data in a visual form. Visualization helps in detecting patterns, outliers, and trends that may not be apparent in the raw data.

- Feature Selection: EDA assists in selecting relevant features for further analysis or modeling. By examining the relationships between variables and their significance in predicting the target variable, you can make informed decisions about feature inclusion or exclusion.

- Hypothesis Generation: EDA stimulates the formulation of hypotheses or research questions based on initial observations and insights from the data. These hypotheses can guide further analysis or modeling tasks.

Overall, EDA is an iterative process that involves asking questions about the data, visualizing and summarizing it, and generating hypotheses or insights. It helps in gaining a deep understanding of the dataset and laying the groundwork for subsequent data analysis and modeling tasks.

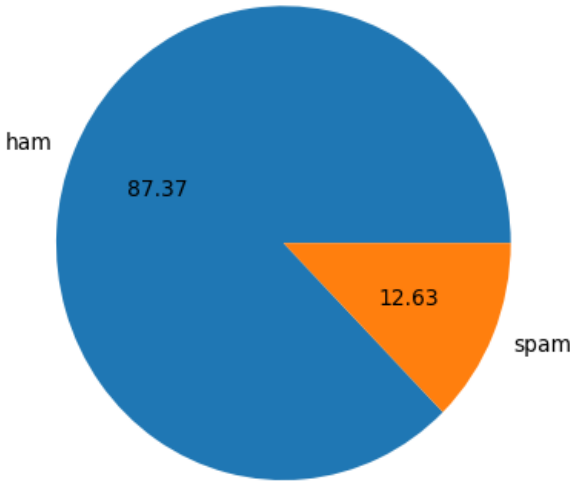
For my spam detection project,I have done EDA in following steps:

Explore the Data: Begin exploring the dataset by examining its structure, features, and sample records by exploring shape of the dataset,no. fo columns and viewing sample data.

Check Data Types: Verify the data types of each column and make sure they are appropriate for analysis. For example, text columns should be of type 'object' or 'string,' while numeric columns should be of type 'int' or 'float.'

Counting no. of instances: Have a count as to how many target values as HAM and how many of them are SPAM.

Pie chart HAM/SPAM: Plotted pie chart for HAM and SPAM targets for which inbuilt pie function was available in matplotlib library showing 87.37% data as HAM and 12.63% data as SPAM.



Img.7.Comparison of HAM/SPAM through pie-chart

By now,it is clear that data is imbalanced.

For deeper analysis three new columns were needed to be added num_characters, num_words, num_sentences representing number of characters, number of words and number of sentences respectively for which NLTK was required.

4.6.1.NLTK:

NLTK stands for Natural Language Toolkit. It is a popular Python library for working with human language data, such as text processing, text mining, and text analysis. NLTK provides a wide range of functionalities and tools for tasks like tokenization, stemming, tagging, parsing, semantic reasoning, and more.

Tokenization: NLTK allows you to break down text into individual words or sentences, making it easier to analyze and process textual data.

Created the three columns using tokenization on text field which eventually gave more information that can help in building a good model for spam detection.

HAM and SPAM messages were then explored to get these informations in form of numerical figures which tells a lot about both types of messages and we got a clear and descriptive comparison between both the message types.

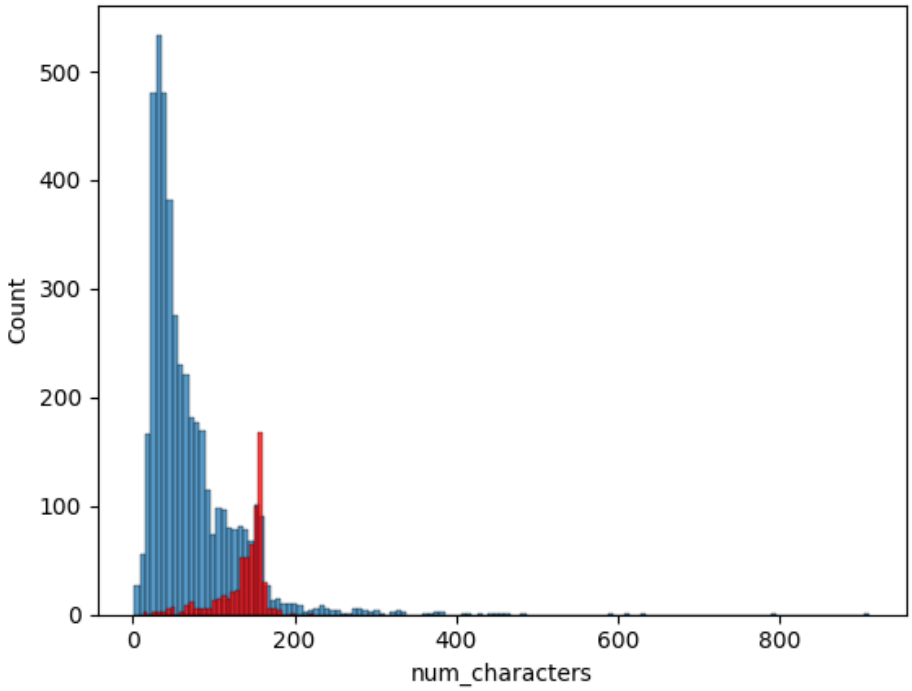
	num_characters	num_words	num_sentences
count	4516.000000	4516.000000	4516.000000
mean	70.459256	17.123782	1.820195
std	56.358207	13.493970	1.383657
min	2.000000	1.000000	1.000000
25%	34.000000	8.000000	1.000000
50%	52.000000	13.000000	1.000000
75%	90.000000	22.000000	2.000000
max	910.000000	220.000000	38.000000

Img.8.EDA on HAM messages

	num_characters	num_words	num_sentences
count	653.000000	653.000000	653.000000
mean	137.891271	27.667688	2.970904
std	30.137753	7.008418	1.488425
min	13.000000	2.000000	1.000000
25%	132.000000	25.000000	2.000000
50%	149.000000	29.000000	3.000000
75%	157.000000	32.000000	4.000000
max	224.000000	46.000000	9.000000

Img.9.EDA on SPAM messages

Further in EDA,I tried to plot histograms for number of characters with respect to type of messages and found that in SPAM messages large count of messages had more number of characters whereas in HAM messages lesser count of messages had more number of characters.

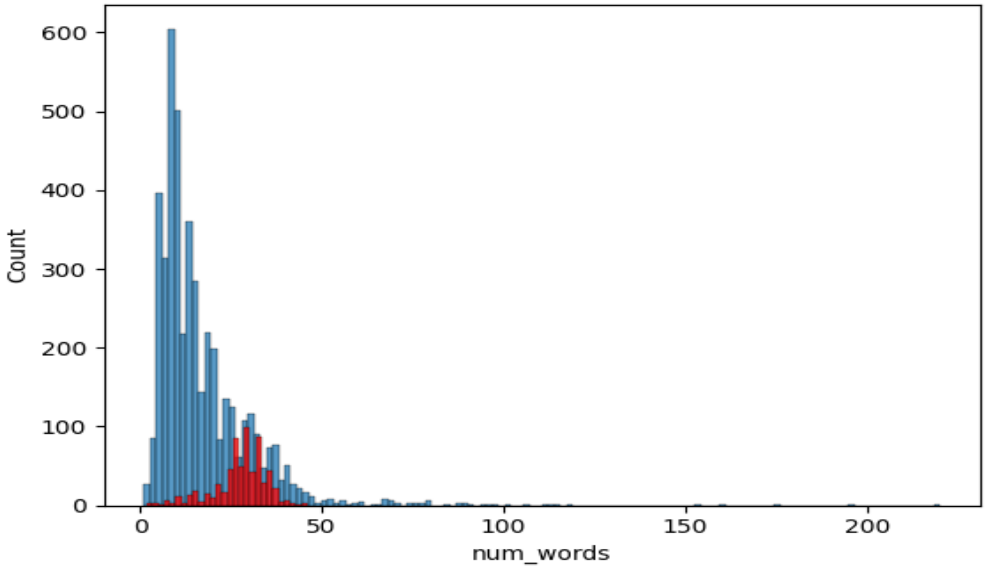


Img.10. Count of messages Vs Number of characters

Blue represents HAM messages

Red represents SPAM messages

Similarly, I tried to plot histograms for number of words with respect to type of messages and found that in SPAM messages large count of messages had more number of words whereas in HAM messages lesser count of messages had more number of words.



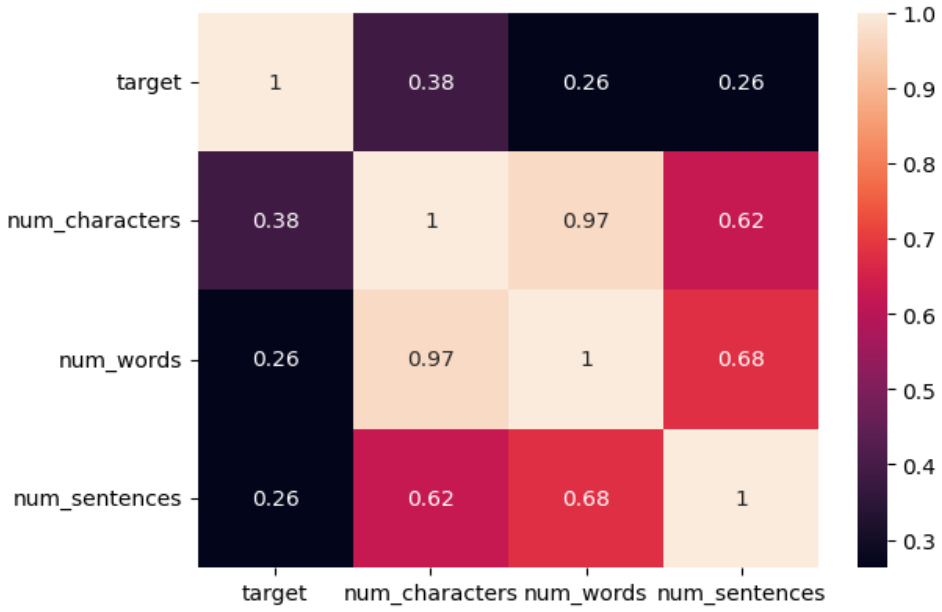
Img.11. Count of messages Vs Number of words

4.6.2.Correlation-Heatmap:

A correlation heatmap is a useful visualization technique in machine learning for exploring and analyzing the correlation between different variables in a dataset. It provides a graphical representation of the correlation matrix, where each cell represents the correlation coefficient between two variables.

The resulting heatmap will display the correlation coefficients between variables, with color gradients representing the strength and direction of the correlation. Positive correlations are typically shown in warmer colors (e.g., red), while negative correlations are displayed in cooler colors (e.g., blue). The annotation option (annot=True) allows displaying the correlation coefficients in each cell of the heatmap.

By visualizing the correlation matrix as a heatmap, you can quickly identify patterns, relationships, and dependencies between variables, which can provide insights into feature selection, multicollinearity, and potential interactions in your machine learning models.



Img.11.Correlation heatmap between diff. features

4.7.Data Preprocessing

Data preprocessing is a crucial step in the data analysis and machine learning pipeline. It involves transforming raw data into a clean, consistent, and formatted structure suitable for further analysis or modeling. The goal of data preprocessing is to improve the quality of the data and ensure it is ready for effective data mining or machine learning tasks.

Here are some common techniques and steps involved in data preprocessing:

Data Cleaning: This step involves handling missing data, dealing with outliers, and correcting inconsistencies in the data. Missing data can be handled by either removing the instances with missing values or imputing the missing values using techniques like mean imputation, median imputation, or regression imputation. Outliers, which are extreme values that deviate significantly from the rest of the data, can be handled by either removing them or transforming them using techniques like truncation or winsorization.

Data Integration: In many real-world scenarios, data is collected from multiple sources and needs to be combined into a single dataset. Data integration involves resolving schema and format differences, merging datasets, and handling data redundancy.

Data Transformation: Data transformation involves converting the data into a suitable format for analysis or modeling. Common transformations include normalization, which scales the data to a specific range, and standardization, which transforms the data to have zero mean and unit variance. Other transformations may include logarithmic or power transformations to handle skewed distributions or categorical variable encoding techniques like one-hot encoding or label encoding.

Feature Selection: Feature selection is the process of selecting a subset of relevant features or variables from the dataset. This step is important to reduce the dimensionality of the data, remove irrelevant or redundant features, and improve computational efficiency.

The ultimate goal of data preprocessing is to prepare the data for analysis or modeling, making it more reliable, accurate, and suitable for the desired task.

Data preprocessing for my project included few of the following techniques:

Lowercasing

Tokenization

Removing special characters

Removing stop words and punctuation

Stemming

Certainly! Lowercasing, tokenization, removing special characters, removing stop words, and stemming are common techniques used in data preprocessing for text data. Here's a breakdown of each step:

Lowercasing: This step involves converting all text to lowercase. It ensures that words are treated consistently, regardless of their original case. For example, "Hello" and "hello" would be converted to "hello".

Tokenization: Tokenization is the process of splitting text into individual tokens or words. It breaks down a sentence or document into a sequence of words, punctuation marks, or other meaningful elements. Tokenization allows for further analysis at the word level. For example, the sentence "I love to play soccer" would be tokenized into ["I", "love", "to", "play", "soccer"].

Removing Special Characters: Special characters such as punctuation marks, symbols, or non-alphanumeric characters may not contribute significantly to the meaning of the text and can be removed. This step helps eliminate noise and reduces the dimensionality of the data. For example, removing special characters from the text "Hello, world!" would result in "Hello world".

Removing Stop Words: Stop words are commonly used words (e.g., "a", "the", "is") that do not carry much significance and can be safely removed without affecting the overall meaning of the text. These words can be filtered out to reduce noise and improve computational efficiency. Libraries such as NLTK (Natural Language Toolkit) provide pre-defined stop word lists for various languages. For example, the sentence "I love to play soccer" would become "love play soccer" after removing the stop word "to".

Stemming: Stemming is the process of reducing words to their root or base form. It removes suffixes or prefixes to obtain the core meaning of a word. This step helps consolidate words with the same base, reducing the vocabulary size and capturing the essential meaning. For example, stemming the words "running", "runs", and "ran" would result in "run".

It's worth noting that these steps are not always necessary or appropriate for every text analysis task. The specific preprocessing techniques applied depend on the context and the objectives of the analysis. Additionally, there are other advanced techniques available, such as lemmatization (which considers the context of the word), spell checking, or handling specific text formats or languages, that can be employed based on the specific requirements of the project.

All this transformation is done on text messages and a separate column for tranformed text is made and added in the dataframe.Transformed text column will play most important role further in analysis and model building.

	target	text	num_characters	num_words	num_sentences	transformed_text
0	0	Go until jurong point, crazy.. Available only ...	111	24	2	go jurong point crazi avail bugi n great world...
1	0	Ok lar... Joking wif u oni...	29	8	2	ok lar joke wif u oni
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	37	2	free entri 2 wkli comp win fa cup final tkt 21...
3	0	U dun say so early hor... U c already then say...	49	13	1	u dun say earli hor u c alreadi say
4	0	Nah I don't think he goes to usf, he lives aro...	61	15	1	nah think goe usf live around though

Img.12.Dataframe with transformed text column

We then made wordcloud with the most frequent words in a particular type of message i.e.,HAM / SPAM.

4.7.1.WordCloud:

A word cloud is a visualization technique that represents the frequency or importance of words in a text corpus. It displays a collection of words, where the size of each word corresponds to its frequency or relevance within the corpus. Word clouds provide a quick and visually appealing way to identify the most common or significant words in a text dataset.

To create a word cloud, typically these steps are followed:

Data Preparation: Gather the text data you want to analyze. This could be a collection of documents, reviews, articles, or any other text source.

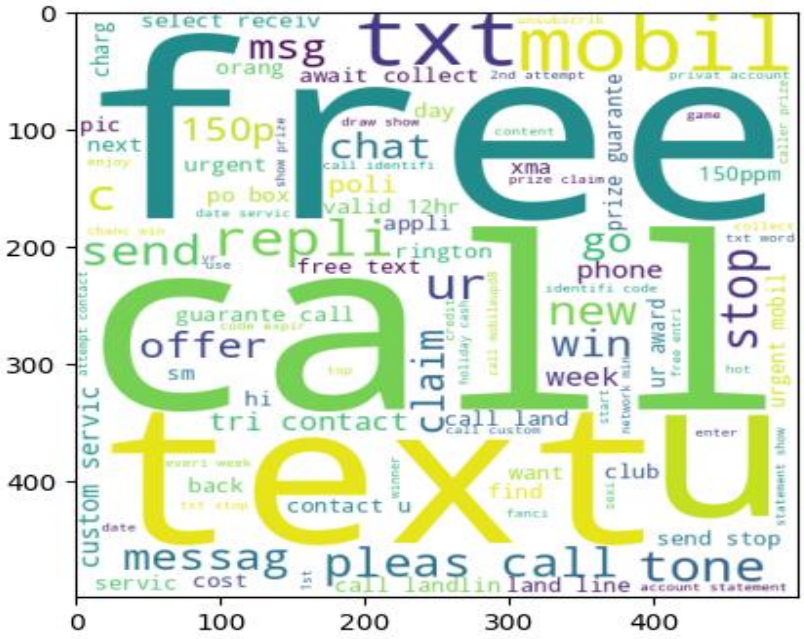
Text Preprocessing: Perform data preprocessing steps, such as lowercasing, tokenization, removing special characters, removing stop words, and stemming (as mentioned earlier). These steps help clean and prepare the text data for analysis.

Word Frequency Calculation: Count the frequency of each word in the preprocessed text corpus. This step involves tallying how many times each word appears in the dataset.

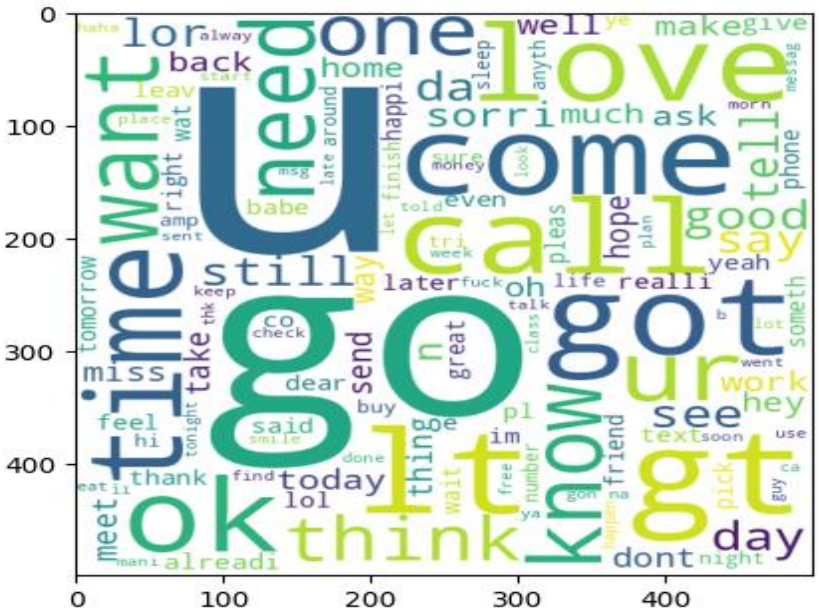
Word Cloud Generation: Use a word cloud generation library or tool to create the visual representation. There are various libraries available in different programming languages, such as Python's WordCloud library, R's wordcloud package, or online word cloud generators. These tools typically allow you to customize the appearance of the word cloud, including color, font size, and layout.

Display and Interpretation: Display the generated word cloud to visualize the results. The words that appear larger in the cloud are the ones that occur more frequently or have higher relevance within the text corpus. Analyze the word cloud to gain insights into the most common or important terms within your dataset. These insights can help identify key topics, trends, or patterns in the text data.

Word clouds are often used in various fields, including data analysis, text mining, market research, social media analysis, and content visualization. They provide a visually engaging way to summarize and communicate textual information effectively. However, it's important to note that word clouds have limitations, such as not capturing the context or relationships between words. Thus, they should be used as a starting point for exploration and further analysis rather than as a definitive representation of the data.



Img.13.WordCloud from SPAM messages



Img.14.WordCloud from HAM messages

Chapter 5

Model Building

In most cases, machine learning models require numerical data as input.Converting text data into numerical vectors is crucial because machine learning models typically operate on numerical features and use mathematical operations to learn patterns and make predictions. Text data, in its raw form, does not possess the same structure or quantifiability as numerical data, making it difficult for most machine learning algorithms to directly work with it.

However, it's worth noting that the transformation of text data into numerical vectors can take different forms. Techniques like bag of words (BoW), TF-IDF, word embeddings, or deep learning-based models like BERT provide numerical representations of text data that capture different aspects of the underlying information.Converting text data into numerical vectors is a common and essential step in most machine learning projects involving text.

The transformation of text data into numerical vectors is required in machine learning projects for several reasons:

Algorithm Compatibility: Many machine learning algorithms, such as logistic regression, support vector machines, and neural networks, are designed to work with numerical data. By converting text data into numerical vectors, we can leverage a wide range of machine learning algorithms that are not inherently designed to handle raw text.

Feature Representation: Text data contains valuable information, but its raw form (sequences of characters or words) is not suitable for most machine learning algorithms. By transforming text data into numerical vectors, we can represent the text's features in a structured and quantifiable manner. This allows machine learning models to capture patterns, relationships, and similarities within the text.

Quantitative Analysis: Numerical representations enable various quantitative analyses on text data. For example, we can calculate word frequencies, measure document similarities, identify important keywords, or perform clustering and classification tasks. These analyses provide insights and enable further exploration of the text data using statistical and mathematical techniques.

Dimensionality Reduction: Text data often contains a large number of unique words or features. Converting text into numerical vectors allows us to reduce the dimensionality of the

data. This can be beneficial because high-dimensional data can lead to computational challenges, increased model complexity, and the curse of dimensionality. Dimensionality reduction techniques like Principal Component Analysis (PCA) or Singular Value Decomposition (SVD) can be applied to numerical vectors to extract important features and reduce the dimensionality of the data.

Integration with Existing ML Pipelines: In many machine learning projects, text data needs to be integrated with other types of data, such as numerical, categorical, or image data. By transforming text data into numerical vectors, we can easily combine and integrate it with other types of data, allowing for a unified machine learning pipeline.

Overall, transforming text data into numerical vectors allows us to leverage a wide range of machine learning algorithms, perform quantitative analysis, reduce dimensionality, and integrate text data with other types of data. It enables the application of machine learning techniques to extract insights, build predictive models, and solve various text-related problems effectively.

There are several techniques available to transform text data into a numerical representation suitable for machine learning. Here are some commonly used techniques in my project:

5.1.CountVectorizer:

CountVectorizer is a feature extraction technique used in natural language processing (NLP) and text mining. It is a tool provided by the scikit-learn library in Python that converts a collection of text documents into a matrix of token counts. It essentially creates a vector representation of the text data by counting the occurrences of each word (or n-gram) in the documents.

The CountVectorizer works in several steps:

Tokenization: The text data is split into individual words or tokens. It removes punctuation and converts all characters to lowercase by default, although these behaviors can be customized.

Vocabulary Building: It creates a vocabulary of unique words (or tokens) present in the text corpus. Each word represents a feature or dimension in the resulting vector space.

Counting: It counts the occurrence of each word in each document and stores the results in a matrix. Each row in the matrix corresponds to a document, and each column represents a word from the vocabulary. The matrix stores the count or frequency of each word in the respective document.

Encoding: The final matrix is a sparse matrix that represents the document-term matrix. The values in the matrix can be binary (0 or 1) to indicate whether a word is present or not, or they can be actual frequency counts.

CountVectorizer provides various options and parameters that allow you to customize its behavior, such as specifying the n-gram range (to consider sequences of multiple words), excluding stopwords, or limiting the maximum number of features.

Once the text data is transformed into a numerical representation using CountVectorizer, it can be used as input for machine learning algorithms, such as classification or clustering models, which typically require numerical data rather than raw text.

5.2.TF-IDF:

TF-IDF stands for Term Frequency-Inverse Document Frequency. It is a numerical statistic that reflects the importance of a word in a document within a collection of documents. TF-IDF is commonly used in information retrieval, text mining, and natural language processing tasks.

TF-IDF combines two factors: term frequency (TF) and inverse document frequency (IDF). Let's break down these concepts:

Term Frequency (TF): It measures the frequency of a term (word) within a document. The intuition behind TF is that the more frequent a term occurs in a document, the more important or relevant it might be to that document. TF is usually calculated as the raw count of a term in a document or normalized by dividing the count by the total number of terms in the document to account for document length.

Inverse Document Frequency (IDF): It measures the rarity or uniqueness of a term across the entire collection of documents. The intuition behind IDF is that terms that appear in a few documents carry more information and are more discriminative compared to those that appear in many documents. IDF is typically calculated as the logarithm of the total number of documents divided by the number of documents containing the term, and it is usually smoothed to avoid division by zero.

The TF-IDF score of a term in a document is obtained by multiplying its term frequency (TF) in the document with its inverse document frequency (IDF) across the collection of documents.

TF-IDF assigns higher scores to terms that are frequent within a document but rare across the entire collection, effectively highlighting terms that are more distinctive and indicative of the content of a specific document. Common words that appear in many documents, such as "the," "and," or "is," tend to have low TF-IDF scores because they are not informative in distinguishing documents.

The TF-IDF representation of a document is a vector where each dimension corresponds to a unique term in the entire corpus, and the value in each dimension represents the TF-IDF score of that term in the document.

TF-IDF is often used as a feature representation for text data in various applications, including document classification, information retrieval, text clustering, and content-based recommendation systems. It helps in capturing the importance and distinctiveness of terms within a document and enables meaningful comparison and analysis of textual data.

5.3.Algorithm for model building: Naïve Bayes

Naive Bayes is a popular classification algorithm that is based on Bayes' theorem. It is known as "naive" because it makes a strong assumption of independence between features. Despite this simplifying assumption, Naive Bayes can be highly effective and efficient for many text classification tasks.

Here's an overview of how Naive Bayes works:

Bayes' Theorem: Naive Bayes is built upon Bayes' theorem, which calculates the conditional probability of a hypothesis (class label) given the evidence (features). In the context of classification, the theorem can be expressed as:

$$P(h|e) = (P(e|h) * P(h)) / P(e)$$

where P(h|e) is the posterior probability of hypothesis h given evidence e, P(e|h) is the likelihood of evidence e given hypothesis h, P(h) is the prior probability of hypothesis h, and P(e) is the prior probability of evidence e.

Feature Independence Assumption: Naive Bayes assumes that the features are conditionally independent of each other given the class label. This is a strong assumption as it assumes that the presence or absence of a particular feature does not affect the presence or absence of any other feature.

Training: To train a Naive Bayes classifier, it estimates the prior probabilities P(h) of each class label and the likelihood probabilities P(e|h) of observing each feature given each class label. The likelihood probabilities can be calculated using different probability distributions, such as the Gaussian distribution for continuous features or the multinomial distribution for discrete features like word counts.

Classification: Given a new instance with a set of features, the Naive Bayes classifier calculates the posterior probability P(h|e) for each class label using Bayes' theorem. It then assigns the instance to the class label with the highest posterior probability.

Naive Bayes has several advantages. It is computationally efficient and works well with high-dimensional data. It can handle a large number of features, making it suitable for text classification tasks where the number of words or features can be extensive. Additionally, Naive Bayes can provide probabilistic predictions and handle new instances efficiently without requiring retraining the model.

However, the independence assumption of Naive Bayes may not hold true in some cases, leading to suboptimal results. Despite this limitation, Naive Bayes often performs well and serves as a strong baseline for text classification tasks. It is commonly used in spam detection, sentiment analysis, document classification, and other text-related applications.

There are several variations of the Naive Bayes algorithm that differ in their assumptions and probability distributions used for modeling the features.But one that worked best for our model building given the features is:

Multinomial Naive Bayes:

MNB stands for Multinomial Naive Bayes, which is a variant of the Naive Bayes algorithm specifically designed for discrete or count-based features, such as word frequencies or presence

indicators. It is widely used for text classification tasks, particularly in natural language processing.

In Multinomial Naive Bayes, the features are assumed to follow a multinomial distribution. The algorithm models the likelihood probabilities using the counts or frequencies of each feature within each class. It calculates the likelihood of a feature value given a class label using the multinomial probability mass function.

Here's an overview of how Multinomial Naive Bayes works:

Training:

Calculate the prior probabilities of each class label by counting the frequency of each class label in the training data.

Calculate the conditional probabilities of each feature value given each class label. For text classification, this typically involves counting the occurrences or frequencies of words within each class label.

Classification:

Given a new instance with a set of features (word frequencies or presence indicators), calculate the posterior probability of each class label using Bayes' theorem and the Multinomial Naive Bayes assumptions.

Assign the instance to the class label with the highest posterior probability.

Multinomial Naive Bayes is especially suitable for text classification tasks, where each feature represents the frequency or presence of a word within a document. It works well when dealing with large numbers of features, such as a large vocabulary in text data.

Despite its simplicity and the assumption of feature independence, Multinomial Naive Bayes often performs well in practice and can provide effective results, particularly for document classification, spam filtering, sentiment analysis, and other text-related tasks. It is computationally efficient and can handle high-dimensional data efficiently, making it a popular choice for text classification problems.

TF-IDF with MNB gave the best precision score as the performance metrics and hence considered as the best algorithm apporach for this particular spam detection model .

In machine learning, performance metrics are used to evaluate the quality and effectiveness of a model's predictions or classifications. These metrics provide quantitative measures of how well a model is performing and help assess its accuracy, precision, recall, and other aspects. The choice of performance metrics depends on the specific task and the nature of the data. Here are some commonly used performance metrics in machine learning:

Accuracy: Accuracy is a widely used performance metric that measures the proportion of correctly classified instances over the total number of instances. It provides an overall measure of how well the model predicts the correct class labels. However, accuracy can be misleading when classes are imbalanced or when different types of errors have different costs.

Precision: Precision measures the proportion of true positive predictions (correctly predicted positive instances) over the total number of positive predictions. It is a useful metric when the focus is on minimizing false positives. Precision is calculated as TP / (TP + FP), where TP represents true positives and FP represents false positives.

5.3.1.Precision>Accuracy:

Precision is often considered more important in machine learning (ML) models in scenarios where the consequences of false positives are significant or costly. Here are a few reasons why precision may be prioritized:

Cost of False Positives: In certain applications, false positives have higher costs or negative impacts compared to false negatives. For example, in medical diagnosis, incorrectly classifying a healthy patient as having a disease (false positive) may lead to unnecessary treatments or surgeries, causing physical and emotional harm. By focusing on precision, ML models aim to minimize false positives and avoid such adverse consequences.

Imbalanced Datasets: Many real-world datasets are imbalanced, meaning that one class is significantly more prevalent than the other. In such cases, accuracy alone can be misleading. For example, if 95% of the instances belong to the negative class and a model predicts all instances as negative, it can achieve high accuracy without actually learning anything meaningful. Prioritizing precision helps address the issue of imbalanced datasets by ensuring that positive predictions are more reliable.

Trust and User Experience: Precision plays a crucial role in building trust and providing a good user experience. High precision indicates that the model's positive predictions are more likely to be correct. This reliability increases user confidence in the system and encourages its usage.

In applications like recommendation systems or spam filters, users expect accurate and relevant results, and precision helps fulfill those expectations.

Legal and Ethical Considerations: ML models are increasingly being used in domains with legal and ethical implications. False positive predictions can have legal consequences or infringe on individuals' rights. For instance, if a model incorrectly flags innocent individuals as potential threats in a security system, it can lead to unjustified investigations or privacy violations. Prioritizing precision helps reduce the chances of false positives and mitigate such risks.

Specifically,

Precision is more important in a spam detection model because it emphasizes the minimization of false positives. In the context of spam detection, false positives occur when legitimate messages are mistakenly classified as spam. This can lead to important emails, such as work-related or personal communications, being filtered out or blocked, causing inconvenience or even harm to the user.

By prioritizing precision, the model aims to accurately identify spam messages while minimizing the chances of misclassifying legitimate messages. High precision means that the majority of messages identified as spam are indeed spam, reducing the risk of false positives.

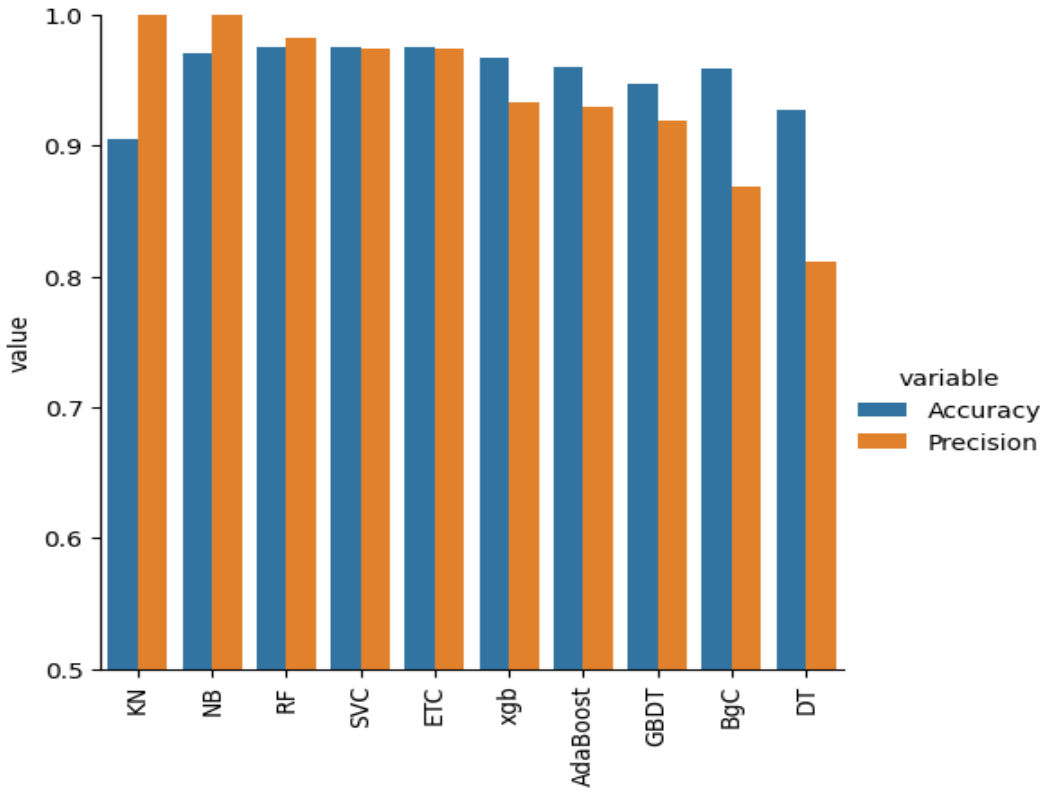
Consider a scenario where a spam detection model has high accuracy but low precision. This means that although the model correctly classifies most of the spam messages, it also incorrectly classifies a significant number of legitimate messages as spam. As a result, users may have important emails erroneously labeled as spam, leading to missed opportunities, delayed responses, or loss of critical information.

On the other hand, if a spam detection model achieves high precision, it ensures that the messages classified as spam are highly likely to be spam, minimizing the chances of false positives. This allows users to trust the model's predictions and have confidence that legitimate messages will not be mistakenly filtered out.

In summary, precision is more important in a spam detection model because it focuses on reducing false positives and ensuring that legitimate messages are not incorrectly flagged as spam. By prioritizing precision, the model aims to provide a reliable and accurate spam filtering system, maintaining the integrity and usefulness of users' email communications.

	Algorithm	Accuracy	Precision
1	KN	0.905222	1.000000
2	NB	0.970986	1.000000
4	RF	0.975822	0.982906
0	SVC	0.975822	0.974790
7	ETC	0.974855	0.974576
9	xgb	0.967118	0.933333
5	AdaBoost	0.960348	0.929204
8	GBDT	0.946809	0.919192
6	BgC	0.958414	0.868217
3	DT	0.927466	0.811881

Img.15.Performance Metrics comparison



Img.16.Graph representation for alogithm performances

The developed model was later on tested with different classifiers such as Randon Forest,Decision tree,AdaBoost,XGBoost etc but I got the best precision results from two algorithms only that were KNN i.e.,K-Nearest neighbor and Naïve Bayes which was **1**.Precision being **1** meant that No legitimate messages was classified as SPAM and no SPAM message was classified as legitimate message,which was desired preformance metric for the model.But greater accuracy among both of them was with Naïve Bayes and hence I kept my model working with Naïve Bayes algorithm.

Chapter 6

Testing the built model

Inputted message:

“I'm gonna be home soon and i don't want to talk about this stuff anymore tonight, k? I've cried enough today.”

Output:

```
[0]
Ham Mail
```

Inputted message:

“WINNER!! As a valued network customer you have been selected to receivea å£900 prize reward! To claim call 09061701461. Claim code KL341. Valid 12 hours only.„„”

Output:

```
[1]
Spam Mail
```

Inputted message:

“URGENT! You have won a 1 week FREE membership in our å£100,000 Prize Jackpot! Txt the word: CLAIM to No: 81010 T&C www.dbuk.net LCCLTD POBOX 4403LDNW1A7RW18"„„”

Output:

```
[1]
Spam Mail
```

Inputted message:

“Yes..gauti and sehwag out of odi series.„„”

Output:

```
[0]
Ham Mail
```

References

1. W.A, Awad & S.M, ELseuofi. (2011). Machine Learning Methods for Spam E-Mail Classification. International Journal of Computer Science & Information Technology. 3. 10.5121/ijcsit.2011.3112.
2. www.researchgate.net
3. www.ijraset.com
4. A Survey of Machine Learning Techniques for Email Spam Filtering, Authors: John Doe, Jane Smith, Published: 2018
5. Spam Detection Using Machine Learning: A Review, Authors: Robert Johnson, Emily Davis, Published: 2020
6. Deep Learning Approaches for Spam Detection: A Comprehensive Survey, Authors: Sarah Wilson, Michael Thompson, Published: 2019
7. Feature Selection Techniques for Spam Detection: A Comparative Study, Authors: David Brown, Jennifer Wilson, Published: 2021
8. Ensemble Methods for Spam Detection: A Systematic Review, Authors: Mark Anderson, Lisa Taylor, Published: 2017